CIS 3300 · INHERITANCE AND POLYMORPHISM, PART 3 OF 4

Abstract Classes, final, and Constructors

Designing the top of a hierarchy, closing it off, and building objects in the right order

---

Parts 1 and 2 used a hierarchy in which every class could be instantiated. This set of notes starts with a new requirement that breaks that design, and fixes it with an *abstract class*. Then it covers the keyword `final` as it applies to classes and methods, and finishes with the rules for constructors in a hierarchy: what runs, in what order, and one trap that catches experienced programmers. For reading, see Core Java, Volume I, Chapter 5, Sections 5.1 and 5.2.

A note before you begin. These are my notes from over the years, cleaned up so they are readable. They are not a study guide, and they are not gospel: not everything here appears in a lecture, and not every topic has notes yet. I write from memory and revise every semester. I also use AI to help convert documents to LaTeX, check for issues, and add explanations. I do not trust it completely, so I am still editing, and a typo or imprecise statement can survive longer than it should. Java also changes over time, though the basics we cover are resilient. If you find something that looks wrong, contradicts the book, or does not behave the way I claim when you run it, tell me. You will be doing the other students a favor, and you will understand the material better for having caught it. The textbook is the tiebreaker, and the compiler beats us both. These notes sometimes go beyond the book, but they do not replace the assigned chapters or the recorded lectures, and neither of those replaces the notes. Read the chapter, watch the lecture, and study the notes.

1. A New Requirement

The company now hires hourly workers. An hourly worker is clearly an employee, so the obvious move is `class HourlyEmployee extends Employee`. But `Employee` has an `annualSalary` field, a `raiseSalary` method, and a `getMonthlyPay` that divides the salary by twelve. None of that fits someone paid by the hour. Inheriting it would give every hourly worker a salary field that means nothing and a raise method that does the wrong thing.

The problem is that `Employee` was doing two jobs: describing what *every* employee has (an ID and a name) and describing one particular way of being paid. The fix is to separate them:

- `Employee` keeps only what all employees share, and says that every employee *can* compute a monthly pay, without saying how.
- `SalariedEmployee` takes over the salary and the raise.
- `HourlyEmployee` holds an hourly rate and the hours worked.
- `Manager` now extends `SalariedEmployee`, and `Executive` still extends `Manager`.

Reorganizing a hierarchy like this is normal. Hierarchies are designed around requirements, and requirements change. Notice, too, that code such as `totalMonthlyPay(ArrayList<Employee>)` from Part 2 keeps working unchanged after the reorganization.

2. Abstract Classes and Abstract Methods

There is no sensible body for `getMonthlyPay` in `Employee` anymore, and there is no such thing as an employee who is "just an employee." Java lets us say both things directly.

```
public abstract class Employee {
    private static int nextId = 1;

    private final String id;
    private String name;

    protected Employee(String name) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("name is required");
        }
        this.id = "E" + nextId;
        nextId++;
        this.name = name.strip();
    }

    public String getId() { return id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public abstract double getMonthlyPay();   // no body: each subclass decides

    @Override
    public String toString() {
        return id + " " + name + ", monthly pay " + String.format("%.2f", getMonthlyPay
    ());
    }

    // equals and hashCode, final and based on id, are unchanged from Part 2.
}
```

An *abstract method* has a header and a semicolon but no body. An *abstract class* is a class that cannot be instantiated. Classes you can instantiate are called *concrete* classes.

```
Employee nobody = new Employee("Nobody");   // does not compile
```

```
error: Employee is abstract; cannot be instantiated
```

The two concrete subclasses each supply the missing body:

```
public class SalariedEmployee extends Employee {
    private double annualSalary;

    public SalariedEmployee(String name, double annualSalary) {
        super(name);
        if (annualSalary < 0) {
            throw new IllegalArgumentException("negative salary: " + annualSalary);
```

```
        }
        this.annualSalary = annualSalary;
    }

    public double getAnnualSalary() { return annualSalary; }

    public void raiseSalary(double percent) {
        annualSalary = annualSalary + annualSalary * percent / 100;
    }

    @Override
    public double getMonthlyPay() {
        return annualSalary / 12;
    }
}
```

```
public class HourlyEmployee extends Employee {
    public static final double REGULAR_HOURS = 160;
    public static final double OVERTIME_RATE = 1.5;

    private double hourlyRate;
    private double hoursThisMonth;

    public HourlyEmployee(String name, double hourlyRate) {
        super(name);
        if (hourlyRate < 0) {
            throw new IllegalArgumentException("rate cannot be negative: " + hourlyRate
    );
        }
        this.hourlyRate = hourlyRate;
    }

    // getters, and a setHoursThisMonth that rejects negative hours, omitted here

    @Override
    public double getMonthlyPay() {
        double regular = Math.min(hoursThisMonth, REGULAR_HOURS);
        double overtime = Math.max(hoursThisMonth - REGULAR_HOURS, 0);
        return regular * hourlyRate + overtime * hourlyRate * OVERTIME_RATE;
    }
}
```

The rules

Table 1: Rules for abstract classes and abstract methods.

Rule	Detail
A class with an abstract method must be abstract	Otherwise some object could exist with a method that has no body.
An abstract class may have no abstract methods	Marking a class abstract is a design choice: "this class exists only to be extended."
A concrete subclass must implement every inherited abstract method	Leaving one out gives "is not abstract and does not override abstract method." A subclass that is itself abstract may leave them for its own subclasses.
An abstract class can have anything else	Fields, constructors, concrete methods, static members. `Employee` still has its ID counter, `toString`, and `equals`.
An abstract class is still a type	`Employee` variables, `Employee[]` arrays, and `ArrayList<Employee>` lists work exactly as before. They just hold objects of concrete subclasses.

Notice how `toString` in the abstract class calls the abstract `getMonthlyPay`. That is perfectly legal, because by the time any `toString` runs, the object is some concrete class that has a body for `getMonthlyPay`. An abstract class can describe the shape of an algorithm and let subclasses fill in the missing steps. You will see this again in the rental example.

3. The final Keyword, Revisited

The review notes introduced `final` on variables and promised two more uses. Here they are.

Table 2: The three meanings of final.

Applied to	Meaning	Example
A variable or field	Can be assigned only once; unrelated to inheritance	`private final String id;`
A method	Cannot be overridden by any subclass	`public final boolean equals(Object other)`
A class	Cannot be extended at all	`public final class Executive extends Manager`

```
public class Chairman extends Executive { }   // does not compile
```

```
error: cannot inherit from final Executive
```

Why close a class or method?

- **To protect a rule.** `Employee.equals` is `final` so that no subclass can change what "the same employee" means (Part 2). The rental example uses a `final` method so that no vehicle can skip the weekly discount.

- **To protect immutability.** Step 4 of the immutable class recipe in the review notes said to make the class `final`. Now you can see why: otherwise someone could write a mutable subclass and pass it anywhere the immutable type is expected. `String` is `final` for exactly this reason, and records are automatically `final`.
- **Because the class was not designed for extension.** Writing a class that others can safely extend takes planning. If you have not done that planning, `final` says so.

4. Constructors in a Hierarchy

Part 1 said that constructors are not inherited, and that every constructor starts by calling a superclass constructor. Here are the full rules.

Every constructor chains upward

- The first statement of a constructor is either `super(...)`, which calls a superclass constructor, or `this(...)`, which calls another constructor in the same class. It cannot be both. A `this(...)` chain must eventually reach a constructor that calls `super(...)`.
- If you write neither, Java inserts `super();` for you. That works only if the superclass has a constructor with no parameters that the subclass can access.
- If a class declares no constructors at all, Java gives it a *default constructor* that takes no parameters and just calls `super()`. As soon as you write any constructor, the default one is no longer generated. This is the usual reason a subclass that used to compile suddenly does not.
- Each class initializes only its own fields. `SalariedEmployee` cannot assign `Employee`'s private `id`; it passes the name up with `super(name)` and lets `Employee` do its own validation.
- Constructors are not polymorphic. The constructor that runs is chosen by the class named after `new`, never by the type of the variable that receives the object.
- Abstract classes have constructors too. `Employee`'s constructor runs every time any kind of employee is created. It is `protected` because only subclasses can call it.

Leaving out `super(name, annualSalary)` in a subclass of `SalariedEmployee` produces this error, which is worth learning to read:

```
error: constructor SalariedEmployee in class SalariedEmployee cannot be applied to
    given types;
  required: String,double
  found:    no arguments
  reason: actual and formal argument lists differ in length
```

"Found: no arguments" is the clue. Java inserted `super()` on your behalf, and no constructor matches it.

Newer Java versions. For many years, the rule was that `super(...)` or `this(...)` had to be the very first statement. Starting with Java 25, through a feature called *flexible constructor bodies*, a constructor may run some statements before `super(...)`, such as checking or preparing an argument, as long as those statements do not use the object being built (no `this`, no instance fields, no instance methods). In Java 24 the feature exists only as a preview that must be turned on. The examples in these notes use the traditional style, which works in every version.

A constructor whose superclass throws a checked exception

If a superclass constructor declares a checked exception, every subclass constructor that calls it must also declare it with `throws`. It cannot catch it instead, because the call to `super(...)` cannot be placed inside a `try` block. We return to checked exceptions in Week 11.

5. The Order of Object Creation

When you write `new Executive(...)`, a lot happens before your constructor's first line of real work. Here is a small program that makes every step visible. `Derived` extends `Base`, each class has a static block, a field initializer that prints a message, and a constructor body.

```
static class Base {
    static {
        System.out.println("Base static block");
    }

    private String baseField = trace("Base field initializer");

    Base() {
        super();   // calls Object(); Java would insert this line for us
        System.out.println("Base constructor body");
        describe();   // risky: this calls an overridable method
    }

    void describe() {
        System.out.println("Base.describe()");
    }
}

static class Derived extends Base {
    static {
        System.out.println("Derived static block");
    }

    private String label = trace("Derived field initializer");

    Derived() {
        super();
        System.out.println("Derived constructor body, label = " + label);
    }

    @Override
    void describe() {
        System.out.println("Derived.describe(), label = " + label);
    }
}

// trace prints its message and returns it.
// main prints "main starts", creates a Derived, prints a divider, and creates another.
```

```
main starts
```

```
Base static block
Derived static block
Base field initializer
Base constructor body
Derived.describe(), label = null
Derived field initializer
Derived constructor body, label = Derived field initializer
(a second object)
Base field initializer
Base constructor body
Derived.describe(), label = null
Derived field initializer
Derived constructor body, label = Derived field initializer
```

Reading the output from top to bottom gives the actual order.

1. **Class initialization, once per class.** The first time a class is used, its static initializers and static blocks run. A superclass is always initialized before its subclass. Notice that the static blocks did not run again for the second object.
2. **Memory for the whole object.** Space for every field, from every level of the hierarchy, is set aside at once and filled with default values: `0`, `false`, or `null`.
3. **The constructor is called, and it immediately calls up the chain.** `Derived()` calls `Base()`, which calls `Object()`.
4. **On the way back down, one level at a time:** after `super(...)` returns, that class's field initializers and instance initializer blocks run, in the order they appear in the file, and then the rest of that constructor's body runs. Then control returns to the next class down, which does the same.

Step 4 is the one that is easy to get wrong. It is not "all field initializers first, then all constructor bodies." `Base`'s field initializer and constructor body both finish before `Derived`'s field initializer starts. Instance initializer blocks, which we saw in the earlier unit, run at the same point as field initializers.

6. Do Not Call Overridable Methods From a Constructor

Look again at the line `Derived.describe(), label = null`. Here is what happened:

1. `Base`'s constructor called `describe()`.
2. The object is a `Derived`, so dynamic binding ran `Derived`'s version, exactly as Part 2 said it would.
3. But `Derived`'s field initializer had not run yet (step 4 above), so `label` still held its default value, `null`.

In this example the result is just a strange line of output. In a real program, the overriding method might call a method on that `null` field and throw a `NullPointerException`, or quietly compute with a zero. The rule:

Rule. A constructor should not call a method that a subclass can override. Call only methods that are `private`, `static`, or `final`.

This rule has a practical side for code you have already written. In the earlier unit, some constructors called their own setters so that validation lived in one place. That is fine in a class nobody extends. In a class that will be extended, either make those setters `final`, or put the validation in a `private` helper method that both the constructor and the setter call.

7. Serialization and a Non-Serializable Superclass

The review notes showed that deserializing an ordinary class does not run its constructor. That is only half the story once inheritance is involved. Suppose a superclass is *not* `Serializable`, but its subclass is:

```
class Person {                                          // not Serializable
    protected String name;

    Person() {                                          // required: a no-argument
    constructor
        System.out.println("  Person() runs");
        name = "(unknown)";
    }

    Person(String name) { this.name = name; }
}

class Worker extends Person implements Serializable {
    private static final long serialVersionUID = 1L;
    private double wage;

    Worker(String name, double wage) { super(name); this.wage = wage; }

    @Override
    public String toString() { return name + " earns " + wage; }
}
```

Saving `new Worker("Ada", 31.5)` and reading it back prints:

```
Before: Ada earns 31.5
Reading back:
  Person() runs
After:  (unknown) earns 31.5
```

Only the serializable part of the object is saved. `wage`, declared in the serializable `Worker`, came back. `name`, declared in the non-serializable `Person`, was never written. To rebuild that part of the object, deserialization calls the no-argument constructor of the first non-serializable superclass, so `name` holds whatever that constructor sets. If `Person` had no accessible no-argument constructor, reading would fail with `java.io.InvalidClassException: Worker; no valid constructor`.

The lesson: if a superclass's fields matter, the superclass needs to be `Serializable` too. `Object` itself is not serializable, which is why this rule always applies at the very top of every hierarchy, harmlessly, since `Object` has no fields.

8. Summary

Table 3: Summary of the ideas in these notes.

Idea	Key point
Abstract method	A header with no body; every concrete subclass must supply one
Abstract class	Cannot be instantiated; can have fields, constructors, and concrete methods; still a type
Concrete class	A class you can instantiate
`final` **method**	Cannot be overridden
`final` **class**	Cannot be extended; used to protect rules and immutability
Constructor chaining	Every constructor calls `super(...)` or `this(...)` first; Java inserts `super()` if you do not
Creation order	Static setup once per class, top down; then per object, each level runs its field initializers and constructor body before the next level down
Constructor trap	Do not call overridable methods from a constructor
Serialization	A non-serializable superclass's fields are not saved; its no-argument constructor runs on reading

9. Try It

1. Add a concrete class `Intern extends Employee` with a fixed monthly stipend, but do not write `getMonthlyPay`. Read the error, then fix it two different ways: once by writing the method, and once by declaring `Intern` abstract. Which fix makes sense here?
2. In `ConstructorOrderDemo`, add an instance initializer block (a bare pair of braces containing a `println`) to `Derived`, above the `label` field. Predict where its message appears in the output, then run it.
3. In `Derived.describe()`, replace the `println` with `System.out.println(label.length());`. What happens when you create a `Derived`, and which rule from these notes explains it?